



VIT[®]
BHOPAL

VELLORE INSTITUTE OF TECHNOLOGY, BHOPAL
SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

A PROJECT REPORT ON

ADAPTIVE TRAFFIC JUNCTION SIMULATOR

A PYTHON-BASED SIMULATION FOR FIXED AND ADAPTIVE TRAFFIC SIGNAL CONTROL

NAME	TEJAL SHARMA
REGISTRATION NO.	26BCE11077
COURSE CODE	CSE1021
COURSE	INTRODUCTION TO PROBLEM SOLVING AND PROGRAMMING
ACADEMIC YEAR	2026 – 2027

PROJECT REPORT

2026 – 2027

02 | TABLE OF CONTENTS

Structured report based on the supplied project archive.

SECTION	TOPIC
03	Introduction & Problem Statement
04	Objectives and Scope
05	System Overview & Architecture
06	Python Modules and Dependencies
07	Important Functions Explained
08	Algorithm
09	Pseudocode
10	Proper Flowchart
11	Traffic Generation Logic
12	Fixed-Time Controller
13	Adaptive Controller Logic
14	Statistics & Performance Metrics
15	Experiment Runner & CSV Handling
16	Important Code Sections
17	Testing & Verification
18	Generated Output
19	Output Analysis
20	Advantages
21	Limitations
22	Future Scope
23	Conclusion
24	References

03 | INTRODUCTION & PROBLEM STATEMENT

The supplied **Adaptive Traffic Junction Simulator** is a Python-based four-way intersection model. Traffic approaches from North, South, East and West. Vehicles are generated stochastically, signal phases control movement, vehicles obey stop-line and spacing rules, and a statistics tracker records performance.

The project compares two signal strategies. The fixed-time controller follows a predetermined cycle, while the adaptive controller observes current queue demand and dynamically chooses the next green phase. The adaptive controller also applies a fairness mechanism when an approach has waited too long.

Problem statement: Develop a modular simulation that represents traffic queues, controls signal phases, measures waiting and queue performance, compares fixed and adaptive control, and supports repeatable experiments with CSV output.

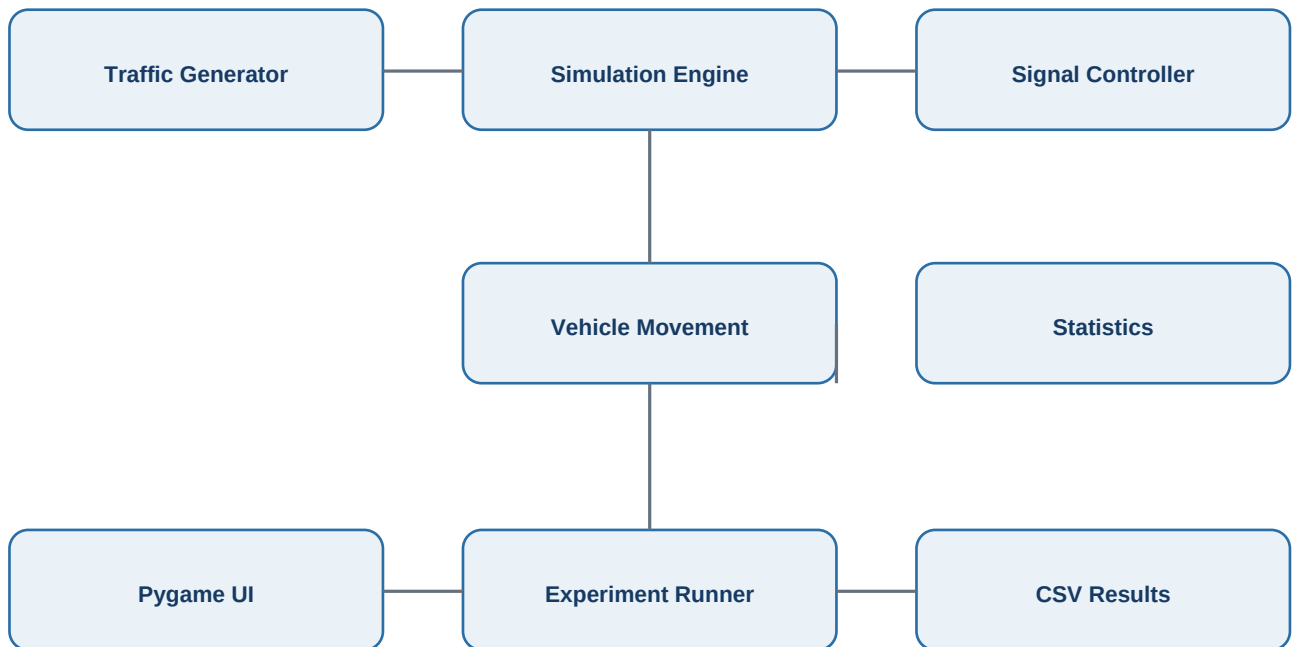
04 | OBJECTIVES AND SCOPE

- Model a four-direction junction with N, S, E and W approaches.
- Generate LOW, MEDIUM and HIGH traffic intensities.
- Compare fixed-time and rule-based adaptive signal control.
- Enforce stop-line, movement and vehicle-spacing constraints.
- Track throughput, waiting time, queue length and vehicles remaining.
- Provide an interactive Pygame visualization and live statistics.
- Support seeded, reproducible headless experiments.
- Export experiment results to CSV.
- Keep simulation, control, UI, statistics and experiment logic modular.

The supplied README defines the project as a simulation rather than a calibrated real-world traffic model. It lists simplified vehicle dynamics, no pedestrian phase, no turning movements, no emergency priority, no real-world road geometry, no sensor noise and synthetic traffic generation as limitations.

05 | SYSTEM OVERVIEW & ARCHITECTURE

The system separates traffic generation, simulation, signal control, vehicle movement, statistics, user interface and experiment execution. The same simulation engine can feed the Pygame application or the headless experiment runner.



Simulation tick order: generate traffic → update signal controller → update vehicle movement → advance time → update statistics.

06 | PYTHON MODULES AND DEPENDENCIES

FILE / MODULE	ROLE
settings.py	Central timing, geometry, traffic intensity, signal phases and UI configuration.
core/simulation.py	Owns simulation state and advances the model one tick at a time.
core/vehicle.py	Represents individual vehicles.
core/movement.py	Handles movement, stop lines, spacing, crossing and exits.
core/traffic_generator.py	Generates seeded stochastic traffic.
core/stats.py	Tracks waiting time, queues, throughput and active vehicles.
control/signal_state.py	Stores current signal phase and remaining time.
control/fixed_controller.py	Implements the fixed signal sequence.
control/adaptive_controller.py	Chooses green phase and duration from queue demand.
ui/app.py	Runs the interactive Pygame application.
ui/renderer.py	Draws the junction and vehicles.
ui/hud.py	Displays controls and live statistics.
experiments/experiment_runner.py	Runs experiment matrices and exports CSV.
requirements.txt	Declares pygame>=2.6.1.

07 | IMPORTANT FUNCTIONS EXPLAINED

FUNCTION	DESCRIPTION	WHY IMPORTANT
TrafficGenerator.generate()	Creates vehicles for each direction using the configured rate, directional weight and time step.	Feeds new traffic into the simulation.
Simulation.step()	Runs the main tick: traffic generation, controller update, vehicle movement, time advance and statistics.	Keeps the complete simulation pipeline synchronized.
Simulation.add_vehicle()	Creates a uniquely identified vehicle, assigns its direction and records its arrival time.	Adds vehicles to the active simulation state.
FixedController.update()	Decreases the remaining phase time and advances the fixed NS/EW signal sequence when a phase ends.	Maintains the deterministic baseline controller.
AdaptiveController._decide_next_green()	Calculates NS/EW demand, checks waiting time, applies the fairness bonus and chooses the next green phase.	Implements the core adaptive decision-making logic.
StatsTracker.update()	Samples queue length, waiting vehicles and related values during every simulation tick.	Builds the performance data used for analysis.
StatsTracker.get_summary()	Returns the current simulation statistics as a structured dictionary.	Provides results to the UI and experiment runner.
App.set_mode()	Replaces the current signal controller with the selected fixed or adaptive controller.	Allows direct comparison between control strategies.
App.set_intensity()	Changes the traffic-generation level between LOW, MEDIUM and HIGH.	Controls the traffic demand used in an experiment.
run_single_experiment()	Runs one complete headless simulation for a selected mode, intensity, seed and duration.	Produces one comparable experiment result.
run_experiment_matrix()	Repeats experiments across the requested seeds, traffic intensities and controller modes.	Automates systematic performance comparisons.
save_results_csv()	Writes experiment dictionaries to a CSV file using csv.DictWriter.	Preserves results for later analysis and charting.

08 | ALGORITHM

1. Load settings and create a seeded traffic generator.
2. Select FixedController or AdaptiveController.
3. Initialize Simulation, SignalState and StatsTracker.
4. Repeat until the selected duration is complete.
5. Generate new vehicles for the current time step.
6. Update the signal controller.
7. Update vehicle movement under signal and spacing constraints.
8. Advance simulation time and update statistics.
9. For adaptive control at ALL_RED, calculate NS and EW queue demand.
10. Apply the fairness bonus when maximum waiting time crosses the threshold.
11. Select the higher-demand axis and calculate bounded green time.
12. Return or display statistics; in experiment mode, write the result to CSV.

09 | PSEUDOCODE

```
START
LOAD settings
CREATE seeded traffic generator
CREATE fixed OR adaptive controller
CREATE simulation

WHILE duration is not complete:
    GENERATE vehicles
    UPDATE controller

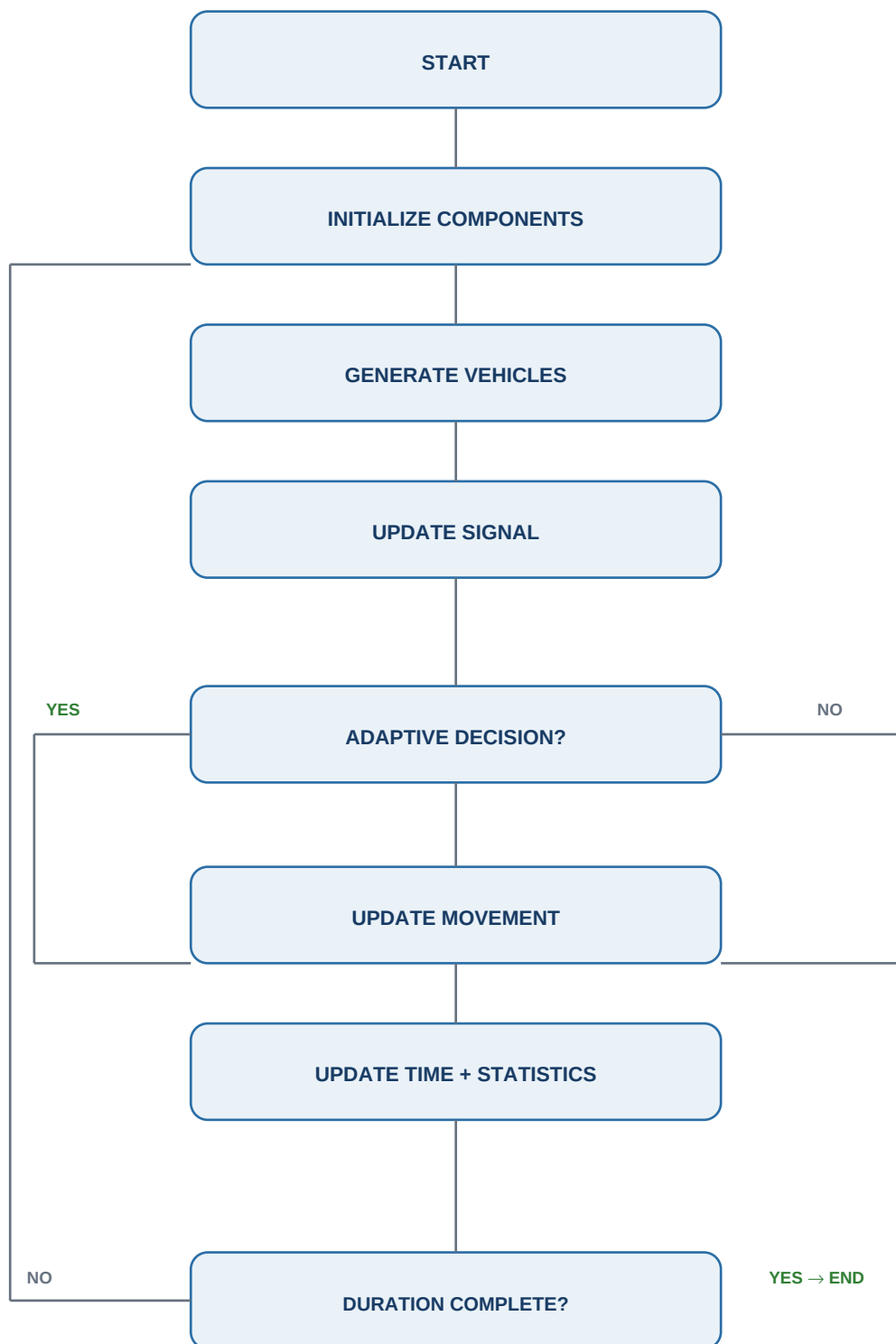
    IF adaptive decision is required:
        NS = N queue + S queue
        EW = E queue + W queue
        CHECK maximum waiting time
        APPLY fairness bonus if threshold is exceeded
        SELECT axis with greater demand
        green = minimum_green + demand * time_per_vehicle
        LIMIT green to maximum_green
        SET selected axis GREEN

    UPDATE vehicle movement
    ADVANCE simulation time
    UPDATE statistics

IF experiment mode:
    STORE result
    WRITE CSV

DISPLAY / RETURN results
END
```

10 | PROPER FLOWCHART



Rounded boxes represent processing steps; decision points are shown as control gates.

11 | TRAFFIC GENERATION LOGIC

INTENSITY	RATE / DIRECTION / SECOND
LOW	0.25
MEDIUM	0.60
HIGH	1.10

TrafficGenerator uses `random.Random(seed)`, making traffic generation repeatable when the same seed is reused. Directional weights can be changed independently.

```
expected = base_rate * weight * dt
count = int(expected)
fraction = expected - count
if rng.random() < fraction:
    count += 1
for _ in range(count):
    simulation.add_vehicle(direction)
```

12 | FIXED-TIME CONTROLLER

The fixed controller follows a predetermined cycle and does not inspect queue lengths when selecting the next green phase.

NS_GREEN → NS_YELLOW → ALL_RED
→ EW_GREEN → EW_YELLOW → ALL_RED

PARAMETER	DEFAULT
Green	6.0 s
Yellow	1.5 s
All-red	0.5 s

13 | ADAPTIVE CONTROLLER LOGIC

The adaptive controller measures queue demand on each axis, checks maximum active waiting time, applies a fairness bonus when required, selects the higher-demand axis and calculates bounded green time.

```
NS demand = N queue + S queue
EW demand = E queue + W queue

green_time = min_green + demand * time_per_vehicle
green_time = min(green_time, max_green)
```

PARAMETER	VALUE
Minimum green	3.0 s
Maximum green	12.0 s
Time per waiting vehicle	0.35 s
Fairness threshold	15.0 s
Fairness bonus	5.0 demand units

14 | STATISTICS & PERFORMANCE METRICS

METRIC	MEANING
Total generated	Vehicles created by the generator.
Total passed	Vehicles that exited the junction.
Average wait time finished	Average waiting time of exited vehicles.
Maximum observed wait	Largest waiting time observed among active or finished vehicles.
Average queue length	Average queue across sampled ticks.
Maximum queue length	Largest queue observed.
Vehicles remaining	Active vehicles at experiment end.
Waiting vehicles	Approaching vehicles stopped at/before the stop line.

15 | EXPERIMENT RUNNER & CSV HANDLING

The experiment runner evaluates controller mode, traffic intensity and random seed combinations without opening Pygame. It returns a flat result dictionary and writes the matrix using `csv.DictWriter`.

```
python run_experiments.py --duration 60
--seeds 42 43 44
--intensities LOW MEDIUM HIGH
--modes fixed adaptive
--output results/experiment_results.csv
```

DIMENSION	OPTIONS
Controller	fixed / adaptive
Intensity	LOW / MEDIUM / HIGH
Seed	User-selectable
Duration	User-selectable; default 60 s
Output	CSV

16 | IMPORTANT CODE SECTIONS

Simulation tick pipeline

Simulation tick pipeline

```
if self.traffic_generator is not None:
    self.traffic_generator.generate(self, dt)

if self.controller is not None:
    self.controller.update(self, dt)
else:
    self.signal_state.update(dt)

movement.update(self, dt)

self.tick += 1
self.time += dt
self.stats.update(self, dt)
```

Adaptive decision

```
if ns_max_wait > fairness_wait_threshold:
    ns_demand += fairness_bonus

if ew_max_wait > fairness_wait_threshold:
    ew_demand += fairness_bonus

next_phase = NS_GREEN if ns_demand >= ew_demand else EW_GREEN
```

CSV export

```
with open(filename, "w", newline="") as file:
    writer = csv.DictWriter(file, fieldnames=RESULT_FIELDS)
    writer.writeheader()
    for row in results:
        writer.writerow(row)
```

17 | TESTING & VERIFICATION

The supplied README states that the repository contains phase-based unit tests and that a verification run executed 54 tests: 52 passed, while two UI-related test modules could not be imported in the documentation runtime because Pygame was not installed there.

ITEM	REPORTED RESULT
Tests executed	54
Tests passed	52
UI import limitation	2 modules; pygame unavailable in that runtime
Headless experiment infrastructure	Executable
Seeded reproducibility	Supported

18 | GENERATED OUTPUT

Actual experiment output generated from the supplied project.

```
Running experiments...

Mode      Intensity  Seed  Dur   Gen  Pass  Rem  AvgWait  MaxWait  AvgQ  MaxQ
-----
fixed     LOW       42    10.0   12   3     9    0.78    6.30    4.49    8
adaptive  LOW       42    10.0   12   5     7    1.52    7.60    4.66    8
fixed     MEDIUM   42    10.0   27   6    21    0.82    6.30    9.87   19
adaptive  MEDIUM   42    10.0   27   9    18    1.13    7.60    9.59   17
fixed     HIGH      42    10.0   37   9    28    1.09    6.30   12.85   23
adaptive  HIGH      42    10.0   37  12    25    1.92    7.60   13.14   22

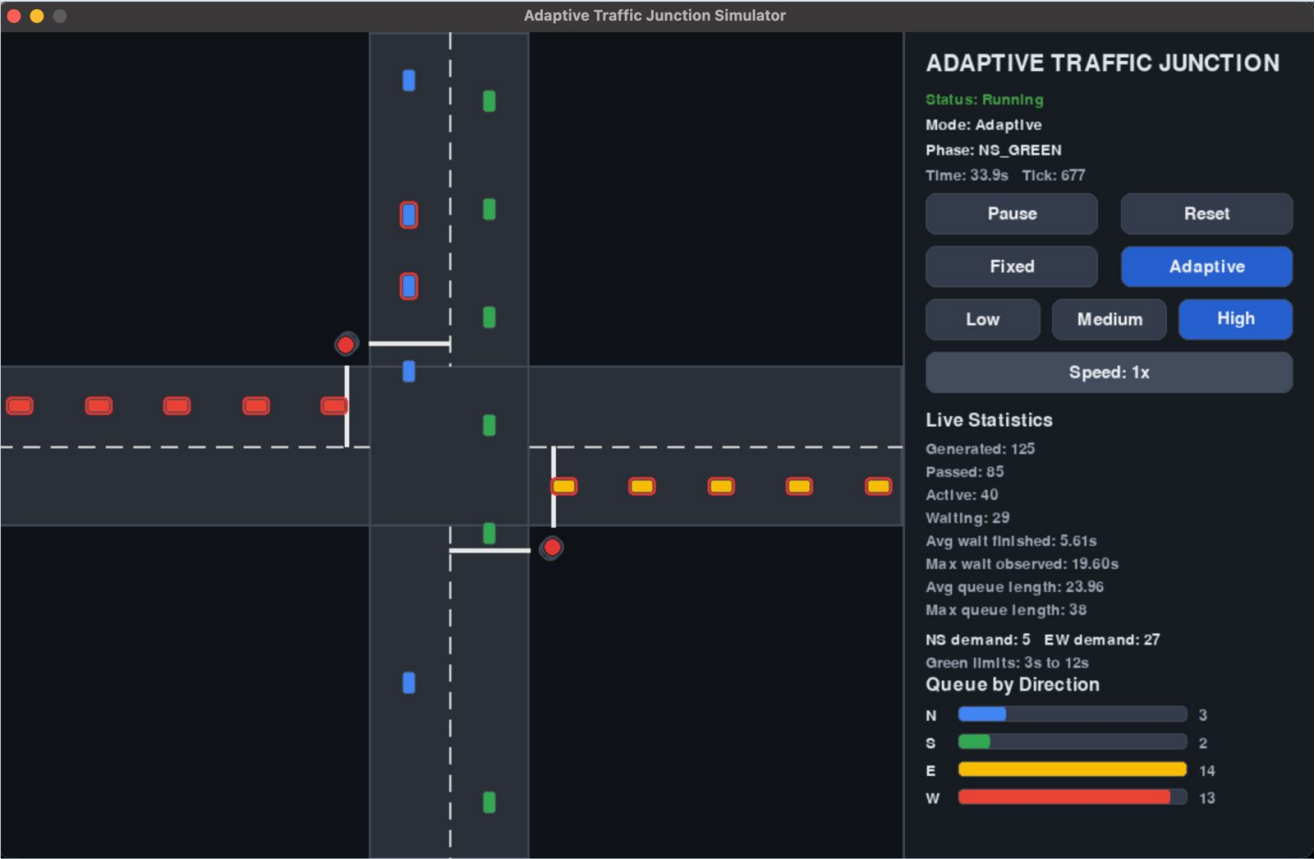
Saved results to: traffic_results.csv
```

The terminal image is a generated verification output, not a user-supplied screenshot. It uses a 10-second duration, seed 42, LOW/MEDIUM/HIGH traffic and both fixed/adaptive modes.

The run generated 12, 27 and 37 vehicles at LOW, MEDIUM and HIGH intensity respectively. For this seed and short duration, adaptive control passed more vehicles in all three displayed intensities.

19 | OUTPUT SCREENSHOT 1

Adaptive control — High traffic intensity



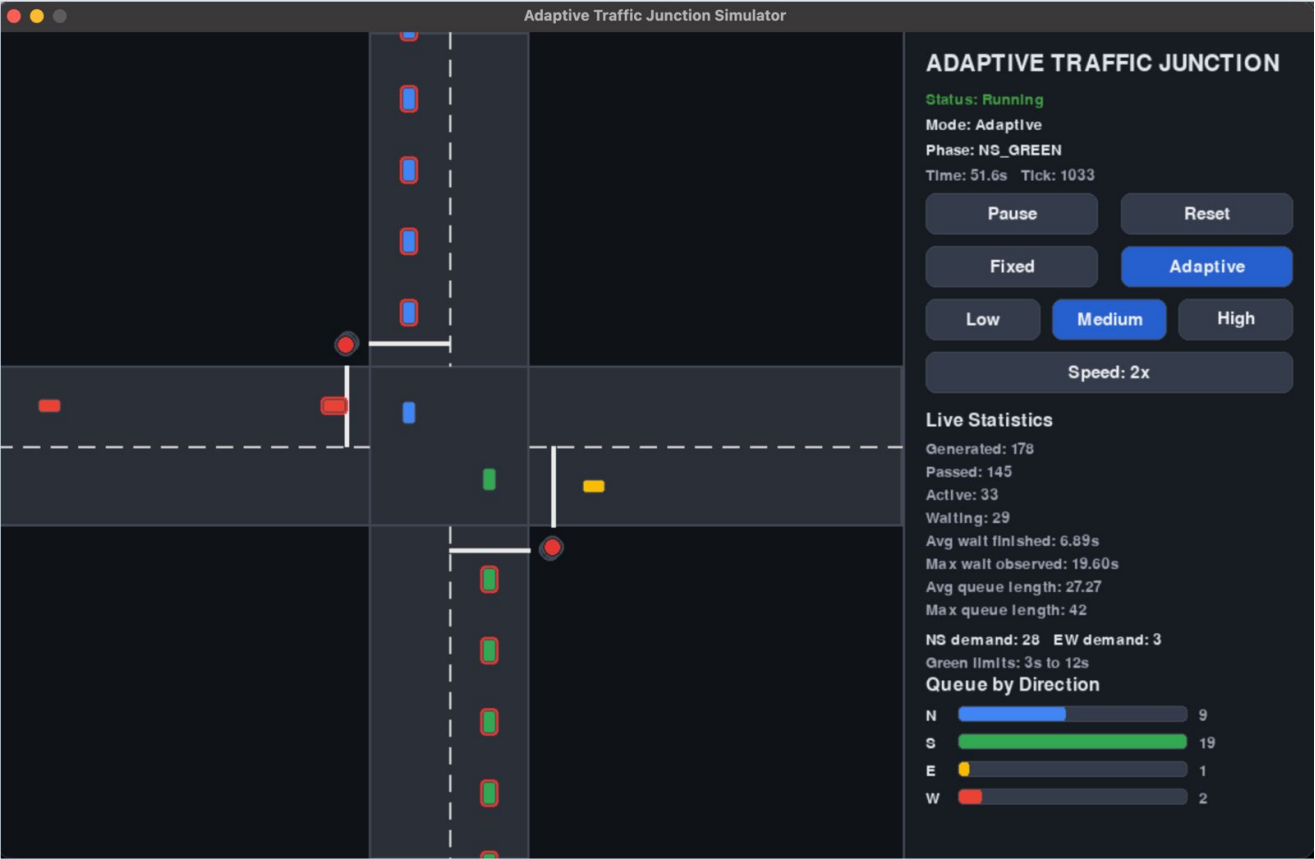
Detailed Analysis

Traffic intensity: HIGH | Mode: Adaptive | Speed: 1x.

The dashboard shows 125 generated vehicles, 85 passed vehicles and 40 active vehicles. The average finished waiting time is 5.61 seconds, while the maximum observed waiting time is 19.60 seconds. The average queue length is 23.96 and the maximum queue length is 38. The live directional queue bars indicate stronger demand on the East and West approaches. This output demonstrates that the adaptive controller is continuously responding to changing traffic demand.

20 | OUTPUT SCREENSHOT 2

Adaptive control — Medium traffic intensity



Detailed Analysis

Traffic intensity: MEDIUM | Mode: Adaptive | Speed: 2x.

The dashboard shows 178 generated vehicles, 145 passed vehicles and 33 active vehicles. The average finished waiting time is 6.89 seconds, while the maximum observed waiting time is 19.60 seconds. The average queue length is 27.27 and the maximum queue length is 42. The directional queue display shows the largest queue on the South approach. This output demonstrates how the simulator changes its traffic state and signal behavior as directional demand changes.

21 | OUTPUT ANALYSIS

INTENSITY	MODE	GEN	PASS	REM	AVG WAIT	AVG QUEUE	MAX QUEUE
LOW	Fixed	12	3	9	0.78 s	4.49	8
LOW	Adaptive	12	5	7	1.52 s	4.66	8
MEDIUM	Fixed	27	6	21	0.82 s	9.87	19
MEDIUM	Adaptive	27	9	18	1.13 s	9.59	17
HIGH	Fixed	37	9	28	1.09 s	12.85	23
HIGH	Adaptive	37	12	25	1.92 s	13.14	22

Observation 1: In this particular run, adaptive control produced more passed vehicles at every displayed intensity.

Observation 2: Average waiting time was higher for adaptive control in this short run, showing that throughput and waiting time can move differently.

Observation 3: At MEDIUM and HIGH intensity, the adaptive maximum queue was slightly lower than the fixed maximum queue.

Important: These are simulation-specific observations. They do not establish that adaptive control is universally superior; results depend on the model, parameters, duration and random seed.

22 | ADVANTAGES

- Modular architecture separates simulation, control, movement, statistics, UI and experiments.
- Fixed and adaptive controllers can be compared using the same engine.
- Seeded randomness supports reproducible experiments.
- Traffic intensity and directional weights are configurable.
- Live statistics provide multiple performance indicators.
- Headless experiment runner supports automated comparisons.
- CSV export makes results easy to analyze later.
- Unit tests are included across development phases.
- Interactive Pygame controls support mode, intensity and speed changes.

23 | LIMITATIONS

- Simplified traffic model rather than a calibrated real-world network.
- Single-lane representation per approach.
- Simplified vehicle dynamics.
- No pedestrian phase.
- No turning movements.
- No emergency-vehicle priority.
- No real-world road geometry.
- No sensor noise or traffic-camera input.
- Synthetic stochastic traffic rather than a calibrated traffic dataset.
- Adaptive control is rule-based rather than machine-learning based.

24 | FUTURE SCOPE

- Add left-turn and right-turn movements and multiple lanes.
- Model pedestrian crossings and dedicated phases.
- Add emergency-vehicle priority.
- Introduce realistic car-following and saturation-flow models.
- Calibrate traffic generation using real traffic datasets.
- Compare additional actuated and optimization-based controllers.
- Add confidence intervals over many random seeds.
- Generate charts automatically from CSV results.
- Add a web visualization layer.
- Evaluate reinforcement-learning signal control as a separate controller.

25 | CONCLUSION

The Adaptive Traffic Junction Simulator demonstrates a complete programming and problem-solving workflow: a traffic model is defined, components are separated into modules, signal-control policies are implemented, statistics are collected, and repeatable experiments are exported for analysis.

The fixed controller provides a deterministic baseline, while the adaptive controller responds to queue demand and includes a fairness mechanism. The simulation engine integrates generation, signal control, vehicle movement and statistics in a predictable tick sequence.

The experiment runner makes the project more than a visual demonstration by enabling systematic comparisons using seeds, traffic intensities and controller modes. The generated verification output also shows why throughput, waiting time and queue length should be interpreted together.

Overall, the supplied project is a strong educational implementation of modular Python programming, simulation, algorithmic decision-making, testing and performance analysis, with clear opportunities for more realistic traffic modeling in future versions.

26 | REFERENCES

- Adaptive Traffic Junction Simulator — supplied project archive and README.md.
- settings.py — supplied project configuration.
- core/simulation.py, core/vehicle.py, core/movement.py, core/traffic_generator.py and core/stats.py.
- control/signal_state.py, control/fixed_controller.py and control/adaptive_controller.py.
- ui/app.py, ui/renderer.py, ui/hud.py, ui/controls.py and ui/theme.py.
- experiments/experiment_runner.py and run_experiments.py.
- tests/ directory supplied with the project.
- requirements.txt — pygame>=2.6.1.

END OF PROJECT REPORT